
AN OVERVIEW OF REINFORCEMENT LEARNING ALGORITHMS

Jin Hyun Park

Dept. of Computer Science and Engineering
Texas A&M University
College Station, TX 77843-3112
jinhyun.park@tamu.edu

1 Preface

This material is intended to provide a comprehensive understanding of the mathematical foundations of reinforcement learning. Unlike other resources that simplify or abbreviate mathematical concepts, this article aims to explore the in-depth mathematical background behind reinforcement learning. It will cover a range of topics, starting with foundational algorithms like Q-learning and SARSA, and progressing to more advanced and recent approaches. Where possible, code implementations for each algorithm will also be included to enhance understanding. Please note that the author is still in the early stages of studying reinforcement learning, having only a few months of self-study in this area. As such, there may be occasional errors or misunderstandings - your understanding and feedback are appreciated!

2 Introduction

If you're an engineering student or work in a technical field, you've probably heard of reinforcement learning at least once. You might be familiar with terms like AlphaGo (for the game of Go) or AlphaStar (for StarCraft II). Even if you haven't, that's okay - there's an easy example to help explain reinforcement learning: the CartPole problem. The CartPole problem is essentially about balancing a pole upright. Think of it as similar to balancing a badminton racket vertically on your palm, except this scenario focuses on a 2D environment where you only need to balance the pole by moving left or right. If the pole starts to tilt left, you move left to stabilize it, and if it tilts right, you move right to keep it balanced. At the early stages of learning, the system struggles to keep the pole balanced. However, as the training progresses, it learns to stabilize the pole effectively. What's fascinating here is that we never explicitly teach the machine how to balance the pole. Instead, we simply reward it for keeping the pole upright longer and penalize it when it fails. With a straightforward goal (balancing the pole) and a well-defined reward system, the machine figures out how to solve the problem without us having to write an algorithm for balancing. Isn't that amazing?

Reinforcement learning can be explained through practical examples like Go, CartPole, or even more complex scenarios like StarCraft and robotics. In Go, players make sequential decisions to place stones strategically, each move affecting the outcome. Similarly, the CartPole problem focuses on balancing a pole by deciding whether to move left or right at each time step to maintain stability. This problem serves as a classic example of reinforcement learning, where the system learns to make sequential decisions to achieve a goal. For more advanced applications, reinforcement learning can determine how to move units and attack opponents in StarCraft II or enable a robot dog to figure out which actions allow it to walk forward. Regardless of the scenario, the core goal of reinforcement learning is the same: selecting the best sequence of actions to achieve the desired outcome.

At its foundation, the primary objective of reinforcement learning is to maximize rewards. The system learns by finding the optimal sequence of actions that leads to the highest cumulative reward. This process naturally raises essential questions: What should the goal be? and When and how should rewards be assigned? Setting clear goals and designing an effective reward system are critical factors that guide the learning process. Together, they form the backbone of reinforcement learning, enabling it to achieve impressive results without explicitly programming every step.

Before diving into the main topic, there's an important distinction to clarify: deep learning and reinforcement learning are fundamentally different. While it might seem like reinforcement learning is a subset of deep learning, they are, in fact, separate concepts. A more accurate way to describe their relationship would be to say, "deep learning is applied

to reinforcement learning." Efforts to integrate deep learning or neural networks into reinforcement learning have existed for a long time, but they were largely unsuccessful until a breakthrough came in 2013. A groundbreaking paper published by Google that year marked a turning point, successfully demonstrating the synergy between deep learning and reinforcement learning. This milestone sparked a surge of research into their collaboration, paving the way for significant advancements in the field. We'll explore this topic in more detail later, but for now, it's essential to keep this distinction in mind as we move forward.

3 Q-Learning

Among the various reinforcement learning algorithms, Q-learning is often considered the most beginner-friendly and a great starting point for understanding the basics. In this section, we will introduce Q-learning along with key concepts such as **greedy action** selection, **epsilon-greedy** strategy, and the **discount factor**. While we will dive deeper into the specifics of the Q-learning algorithm later, this section focuses on providing a foundational understanding of reinforcement learning through simple examples. By the end, you'll have a clearer idea of how reinforcement learning operates and how Q-learning fits into the bigger picture.

3.1 Greedy Action

In Q-learning, the agent assigns scores to different actions as it moves through states (it updates the board as the agent moves), and it selects the action with the highest score. This is known as the **greedy action**. More specifically, Q-learning calculates scores for each state-action pair. For example, if the agent selects the right action from state 1 and reaches state 2, where the highest value is x , then the Q-value for $(state_1, right)$ is updated to x . That is, $Q(state_1, right) = x$. We'll delve into those details later. For now, let's use an example to understand Q-learning with greedy action, as shown in Figure 1.

The agent starts in the top-left corner, represented by the green square. The agent can move one step at a time in one of four directions: up, down, left, or right (equivalent to north, south, west, or east). Each square has a score (initially set to 0), except for the orange squares, which represent the goals. Reaching a goal ends the current episode. In this example, there are two goals: one offers a reward of 1, and the other offers a reward of 10. Any square without a specified score can be assumed to have a score of 0.

Now, suppose the agent begins its journey and somehow reaches the top-right goal with the reward of 1. In the beginning, when all values are zero, the agent has no option but to move randomly. Just before reaching the goal, the agent selects the "up" action, so the square just below the goal gets assigned a score of 1 (equal to the goal's reward). This updated board becomes the second state. Next, the agent starts over from the initial position (green) and eventually reaches the square (this square is not a goal square) with a score of 1 (moving right to reach it). Here, we say the score is one because 1 is the highest value among all four values $(N, S, W, E) = (1, 0, 0, 0)$ is one. This updated state becomes the third board.

By repeating this process, the agent learns and eventually discovers a path to the top-right goal. Once this happens, the agent consistently follows the path that yields the highest reward. What behavior do we see from the agent? It always moves toward the action with the highest reward. For example, if the state's Q-values for the four possible directions (N, W, S, E) are $(1, 0, 0, 0)$, the agent will always move north. This is the essence of the greedy action: the agent always selects the best immediate option.

However, is this approach truly optimal? The agent might miss better paths or higher rewards because it only focuses on the known best actions and doesn't explore other possibilities. For example, there might be a faster route or a better goal that the agent completely overlooks. It is clear that moving three steps to the right is a more efficient path, but since the agent follows a greedy approach (i.e., it selects an action with a value of 1 always), it will never explore this better route on its own. To address this inefficiency, **epsilon-greedy** strategy and the **discount factor** come into play. These techniques balance exploration and exploitation, ensuring the agent explores other possibilities while still striving to maximize rewards.

3.2 Epsilon-greedy (Exploration)

Epsilon (ϵ) is a value between 0 and 1. For example, if $\epsilon = 0.1$, the agent moves randomly with a probability of 0.1 (10%), while in the remaining 90% of cases, it follows a greedy policy. If $\epsilon = 1$, the agent moves entirely at random, maximizing **exploration**. Conversely, if $\epsilon = 0$, there is no randomness at all, maximizing **exploitation**. The term exploitation here essentially means "making the best use of the numbers already present on the board." Ideally, the agent should strike a balance between exploration and exploitation, as there is a trade-off between the two. What are the

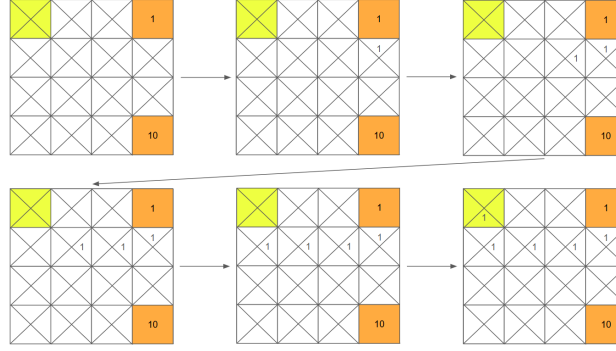


Figure 1: Visualization of Q-learning with a greedy policy. The agent (yellow) starts in the top-left corner and learns by propagating rewards from goal states (orange). Due to the greedy strategy, the agent consistently follows the highest immediate reward path, ignoring the more optimal goal with a reward of 10.

advantages of exploration? Firstly, it helps the agent discover **new paths**. Secondly, it allows the agent to find **new goals**. Each case is illustrated in Fig. 2, where the left side represents the first scenario, and the right side represents the second. When using the epsilon-greedy method, it is common to apply a **decaying epsilon-greedy** approach. Decaying

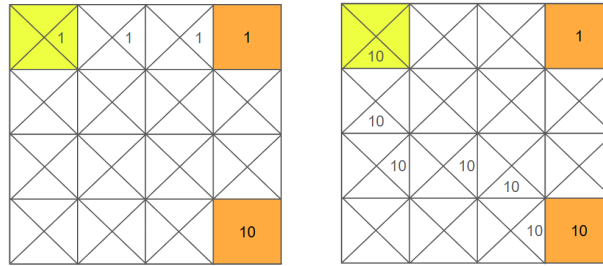


Figure 2: The effect of exploration in Q-learning. Compared to Fig. 1, the left grid demonstrates how exploration allows the agent to discover a shorter path to the goal with a reward of 1. The right grid further highlights the benefit of exploration by enabling the agent to find a higher reward goal (10). This illustrates the importance of balancing exploration and exploitation to improve learning efficiency.

epsilon refers to gradually reducing the initial epsilon value ϵ over time. For example, if the initial epsilon value is 0.5, the agent will explore more frequently in the early episodes. As the episodes progress, the epsilon value gradually decreases towards 0, leading to reduced exploration and increased exploitation. This approach ensures that the agent explores sufficiently at the beginning while gradually focusing on optimizing its learned policy (i.e., path) as it gains more experience.

3.3 Discount Factor

You might have already noticed that the path in Fig. 1 and the left path in Fig. 2 are different. The left path in Fig. 2 appears more reasonable than the one in Fig. 1. However, at the start, there is no difference between choosing $Q(state_{start}, right)$ and $Q(state_{start}, down)$ because both have a value of 1. Ideally, choosing $Q(state_{start}, right)$ should be recognized as the better option, and this should be properly reflected on the board. The discount factor (γ) can help solve this issue. It encourages the agent to find a more efficient path. While the discount factor is often introduced for mathematical convergence, we will explore that aspect later.

The discount factor (γ) is also a value between 0 and 1. Remember how we previously assigned a value of 1 to a specific action when the next state had a value of 1? Instead of simply duplicating that value, we now multiply it by gamma (γ), which helps differentiate between immediate and long-term rewards. For a clearer understanding, refer to Fig. 3.

We have established that the discount factor (γ) helps the agent find a more **efficient path**. Secondly, γ also allows the agent to **balance current vs. future rewards**. If γ is high, the agent places greater importance on future rewards, leading to long-term planning. Conversely, if γ is low, the agent focuses more on immediate rewards and considers future rewards less significant. This trade-off influences the agent's decision-making strategy and learning behavior.

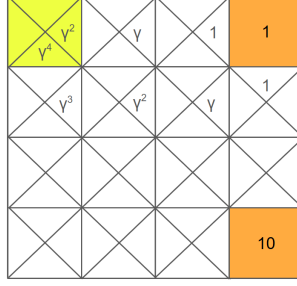


Figure 3: Visualization of the discount factor (γ) in Q-learning. As the agent moves away from the goal (orange), the reward is discounted by powers of γ , encouraging the selection of more efficient paths. This prevents the agent from treating all rewards equally and helps it prioritize shorter, higher-reward routes.

3.4 Q-update

Let's conclude this section by considering the most efficient path in Fig. 3. So far, we have explored Q-learning through simple examples, but we have not yet discussed the Q-learning update rule in detail. Previously, when explaining reward propagation, we described how values were updated instantaneously as γ , γ^2 , γ^3 , and so on. However, in reality, the update process is not direct but rather soft and gradual. The actual update method follows the following equation,

$$Q(s_t, a_t) \leftarrow (1 - \alpha) \cdot Q(s_t, a_t) + \alpha \cdot (R_t + \gamma \max_{a_{t+1}} Q(s_{t+1}, a_{t+1})) \quad (1)$$

where s represents the state, a is the action, R represents the reward, and t is the current time step. In fact, the update approach we used earlier can be seen as an extreme case where the learning rate (α) was set to 1. Typically, α is a value between 0 and 1, similar to the learning rate in deep learning. The alpha (α) value represents how much new information is incorporated into the learning process. When α is high, the board values change drastically with each update, relying less on past values. When α is low, the updates are smaller, meaning the board values evolve more gradually while incorporating more information from past experiences. Thus, selecting an appropriate learning rate (α) is crucial for balancing the learning process, ensuring stable and efficient convergence.

4 Markov Decision Process (MDP)

In the context of a Markov Decision Process (MDP), a *decision* refers to the action that the agent takes at each state. The process consists of a sequence of actions taken over time, which is why it is referred to as a *process*. However, what makes it a *Markov* process? This can be analyzed using Fig. 3 as a reference.

Assume the agent starts at state s_0 and takes an action a_0 , resulting in a transition to a new state s_1 (which could potentially be the same as s_0). At this point, the agent selects a new action a_1 , leading to state s_2 , and this process continues iteratively. The Markov property states that the future state depends only on the current state and action, not on past states or actions. This leads to two fundamental equations.

4.1 Action Probability Dependence

$$p(a_1 \mid s_0, a_0, s_1) = p(a_1 \mid s_1) \quad (2)$$

Since s_1 is determined by s_0, a_0 , there is no need to consider them when selecting a_1 . The probability distribution $p(a_1 \mid s_1)$ represents the likelihood of taking action a_1 given state s_1 . In Fig. 3, this corresponds to the fact that the agent has a certain probability of moving in one of four directions: up, down, left, or right. This probability distribution over actions for a given state is defined as the *policy*, denoted as $\pi(a_t \mid s_t)$. If a_1, s_0, a_0, s_1 are continuous variables, then $p(a_1 \mid s_1)$ is a probability density function (PDF). Conversely, if they are discrete, then it is a probability mass function (PMF).

4.2 State Transition Probability

$$p(s_2 \mid s_0, a_0, s_1, a_1) = p(s_2 \mid s_1, a_1) \quad (3)$$

The next state s_2 depends only on the previous state s_1 and the action a_1 taken in that state. Therefore, we do not have to consider s_0 and a_0 . The probability distribution $p(s_t \mid s_{t-1}, a_{t-1})$ is referred to as the *transition probability*. Thus,

the Markov property ensures that decisions and state transitions depend solely on the present state and action without requiring knowledge of past states, thereby making computations more efficient.

At this point, we have covered the fundamentals of the Markov Decision Process (MDP). In reinforcement learning, the ultimate goal is to maximize the reward. More precisely, the objective is to maximize the *return*. To be even more specific, the goal is to maximize the *expected return*. In the next section, we will explore the concept of return in detail.

5 Return

The definition of return is given by:

$$G_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \dots \quad (4)$$

$$= R_t + \gamma G_{t+1} \quad (5)$$

Return (G_t) represents the summation of discounted rewards over time. Here, R_t denotes the reward received when action a_t is taken in state s_t . In other words, an agent located in state s_t at time t takes an action a_t and receives a reward R_t . The return G_t is a weighted sum of rewards, where each future reward is multiplied by the discount factor γ , which determines the relative importance of future rewards.

But why do we refer to this as the *expected return*? The reason is that both states and actions are random variables, making the return inherently stochastic. Since reinforcement learning aims to maximize return, the objective is to find the optimal action that achieves this maximization. More formally, the goal is to determine the optimal policy that maximizes the expected return. Maybe you are confused about a discount factor. See the next section for more details about it.

6 Discount Factor (γ)

The discount factor γ determines the significance of immediate rewards relative to future rewards in an agent's learning process. If γ is small, the agent exhibits myopic behavior, prioritizing actions that yield immediate rewards while largely ignoring future consequences. In this case, reward propagation becomes challenging due to the diminished impact of delayed rewards. Conversely, if γ is large, the agent considers long-term rewards, allowing future rewards to influence decision-making. When γ is high, reward propagation occurs more effectively, facilitating a broader evaluation of potential actions. According to prior discussions [1], the discount factor γ can be interpreted as defining a *time horizon*. This relationship can be formally expressed as:

$$G_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \dots \quad (6)$$

$$= \sum_{k=0}^{\infty} \gamma^k R_{t+k} \quad (7)$$

$$= \sum_{\Delta t=0}^{\infty} e^{-\Delta t/\tau} R_{t+\Delta t} \quad (8)$$

where $\gamma = e^{-1/\tau}$ and $k = \Delta t$. From this formulation, it is evident that the time horizon τ is directly related to the discount factor γ . For instance, when $\gamma = 1$, the time horizon τ approaches infinity, implying that rewards propagate indefinitely across time steps.

7 State value function, Action value function, and Optimal policy

The goal of reinforcement learning is to maximize the expected return, and it does so by discovering the actions or, more precisely, the policy that yields this maximum. In other words, reinforcement learning seeks the policy that prescribes which actions to take in order to maximize the expected return.

In this section, we delve deeper into the concept of *expected return*. There are two main ways to represent expected return mathematically:

1. State Value Function
2. Action Value Function

Before defining these formally, let's build some intuition about the state value function. The state value function represents the expected return starting from the current state onward. In other words, it is the *average return* that an agent can expect to accumulate from this state forward. Conceptually, you can think of the state value function as an evaluation or "value assignment" of the current state. Naturally, having a high state value is desirable, and from a more precise standpoint, we want to maximize that value.

The action value function, on the other hand, assigns a value (or "score") to each possible action. In a previous section, we introduced Q-learning, where we assigned values to actions and discovered a path toward the goal. That Q value corresponds exactly to the action value function. Intuitively, it indicates the return that can be expected from taking a given action in a given state.

Finally, the optimal policy is the policy that maximizes the expected return from this point onward. In other words, the optimal policy maximizes the state value function. One important note is that the optimal policy focuses on maximizing future rewards from the current state forward. It does not consider past states or rewards once the current state is reached. We will now begin by examining the state value function in detail.

7.1 State Value Function

The state value function is an expectation of G_t given the state s_t . This can be defined as below:

$$V(s_t) \triangleq \int_{a_t: a_\infty} G_t \cdot p(a_t, s_{t+1}, a_{t+1}, \dots | s_t) da_t : a_\infty \quad (9)$$

Recall that the *optimal policy* is the one that maximizes the state value function $V(s_t)$. Formally, we can write:

$$\pi^*(a_t | s_t) = \arg \max_{\pi} V(s_t)$$

where π denotes a policy and $*$ indicates the optimal case for every state s_t . Note that the expectation of x and $f(x)$ is defined as below.

$$E[x] = \int_x x \cdot p(x) dx, \quad x \sim p(x) \quad (10)$$

$$E[f(x)] = \int_x f(x) \cdot p(x) dx, \quad x \sim p(x) \quad (11)$$

7.2 Action Value Function

The action value function is an expectation of G_t given the state s_t and action a_t

$$Q(s_t, a_t) \triangleq \int_{s_{t+1}: a_\infty} G_t \cdot p(s_{t+1}, a_{t+1}, s_{t+2}, \dots | s_t, a_t) ds_{t+1} : a_\infty \quad (12)$$

7.3 Bellman Equation (Advanced Reader Only)

It is not strictly necessary to understand the Bellman equation, but it appears frequently in research articles. By using the Bellman equation, we can express the state value function V in terms of the action value function Q and vice versa. In other words, it becomes possible to describe V using Q , or Q using V . Moreover, we can write $V(s_t)$ in terms of $V(s_{t+1})$, or $Q(s_t, a_t)$ in terms of $Q(s_{t+1}, a_{t+1})$. From my understanding, this approach of expressing V in terms of the next V and Q in terms of the next Q is essentially what we call the Bellman equation. In this subsection, we will discuss: (1) expressing V in terms of Q , (2) V in terms of the next V , (3) Q in terms of V , and (4) Q in terms of the next Q .

• Representing V by using Q

Using Bayes' rule, the term

$$p(a_t, s_{t+1}, a_{t+1}, \dots | s_t)$$

in the state value function can be rewritten as

$$p(s_{t+1}, a_{t+1}, \dots | s_t, a_t) \cdot p(a_t | s_t).$$

By substituting

$$p(a_t, s_{t+1}, a_{t+1}, \dots | s_t)$$

with

$$p(s_{t+1}, a_{t+1}, \dots \mid s_t, a_t) \cdot p(a_t \mid s_t),$$

we obtain

$$V(s_t) \triangleq \int_{a_t} \int_{s_{t+1}:a_\infty} G_t \cdot p(s_{t+1}, a_{t+1}, \dots \mid s_t, a_t) ds_{t+1} : a_\infty p(a_t \mid s_t) da_t. \quad (13)$$

This expression simplifies further, showing that $V(s_t)$ can be interpreted as the expectation of $Q(s_t, a_t)$ with respect to the action a_t .

$$V(s_t) \triangleq \int_{a_t} Q(s_t, a_t) \cdot p(a_t \mid s_t) da_t \quad (14)$$

Upon examining this equation, it appears quite reasonable. In our previous grid world example, each cell had four possible Q-values corresponding to the actions of moving up, down, left, or right. The essence of this equation is that the *average* of those Q-values (weighted by the probability of choosing each action) is precisely $V(s_t)$.

- **Representing V by Using Next V**

Using the Bayesian rule again,

$$p(a_t, s_{t+1}, a_{t+1}, \dots \mid s_t)$$

in the state value function can be converted into

$$p(a_{t+1}, \dots \mid s_t, a_t, s_{t+1}) \cdot p(a_t, s_{t+1} \mid s_t).$$

By employing the Markov Decision Process (MDP) assumption, this further simplifies to

$$p(a_{t+1}, \dots \mid s_{t+1}) \cdot p(a_t, s_{t+1} \mid s_t).$$

Therefore, substituting

$$p(a_t, s_{t+1}, a_{t+1}, \dots \mid s_t)$$

with

$$p(a_{t+1}, \dots \mid s_{t+1}) \cdot p(a_t, s_{t+1} \mid s_t)$$

yields:

$$V(s_t) \triangleq \int_{a_t, s_{t+1}} \int_{a_{t+1}:a_\infty} (R_t + \gamma G_{t+1}) \cdot p(a_{t+1}, \dots \mid s_{t+1}) da_{t+1} : a_\infty p(a_t, s_{t+1} \mid s_t) da_t, s_{t+1} \quad (15)$$

$$\triangleq \int_{a_t, s_{t+1}} (R_t + \gamma V(s_{t+1})) \cdot p(a_t, s_{t+1} \mid s_t) da_t, s_{t+1} \quad (16)$$

$$\triangleq \int_{a_t, s_{t+1}} (R_t + \gamma V(s_{t+1})) \cdot p(s_{t+1} \mid s_t, a_t) \cdot p(a_t \mid s_t) da_t, s_{t+1}. \quad (17)$$

Typically, $p(s_{t+1} \mid s_t, a_t)$ is called the *transition probability*, and $p(a_t \mid s_t)$ is called the *policy*.

- **Representing Q by Using V**

Using the Bayesian rule again, the term

$$p(s_{t+1}, a_{t+1}, \dots \mid s_t, a_t)$$

in the action value function can be rewritten as

$$p(a_{t+1}, s_{t+2}, \dots \mid s_t, a_t, s_{t+1}) \cdot p(s_{t+1} \mid s_t, a_t).$$

By applying the Markov Decision Process (MDP) assumption, this further simplifies to

$$p(a_{t+1}, s_{t+2}, \dots \mid s_{t+1}) \cdot p(s_{t+1} \mid s_t, a_t).$$

Substituting

$$p(s_{t+1}, a_{t+1}, \dots \mid s_t, a_t)$$

with

$$p(a_{t+1}, s_{t+2}, \dots | s_{t+1}) \cdot p(s_{t+1} | s_t, a_t)$$

yields the following equation:

$$Q(s_t, a_t) \triangleq \int_{s_{t+1}} \int_{a_{t+1}:a_\infty} (R_t + \gamma G_{t+1}) \cdot p(a_{t+1} : a_\infty | s_{t+1}) da_{t+1} : a_\infty p(s_{t+1} | s_t, a_t) ds_{t+1} \quad (18)$$

$$\triangleq \int_{s_{t+1}} (R_t + \gamma V(s_{t+1})) \cdot p(s_{t+1} | s_t, a_t) ds_{t+1}. \quad (19)$$

• Representing Q by Using Next Q

Using the Bayesian rule again, the term

$$p(s_{t+1}, a_{t+1}, \dots | s_t, a_t)$$

in the action value function can be rewritten as

$$p(s_{t+2} : a_\infty | s_t, a_t, s_{t+1}, a_{t+1}) \cdot p(s_{t+1}, a_{t+1} | s_t, a_t).$$

By applying the Markov Decision Process (MDP) assumption, this expression simplifies to

$$p(s_{t+2} : a_\infty | s_{t+1}, a_{t+1}) \cdot p(s_{t+1}, a_{t+1} | s_t, a_t).$$

Substituting

$$p(s_{t+1}, a_{t+1}, \dots | s_t, a_t)$$

with

$$p(s_{t+2} : a_\infty | s_{t+1}, a_{t+1}) \cdot p(s_{t+1}, a_{t+1} | s_t, a_t)$$

yields:

$$Q(s_t, a_t) \triangleq \int_{s_{t+1}, a_{t+1}} \int_{s_{t+2}:a_\infty} (R_t + \gamma G_{t+1}) \cdot p(s_{t+2} : a_\infty | s_{t+1}, a_{t+1}) ds_{t+2} : a_\infty p(s_{t+1}, a_{t+1} | s_t, a_t) ds_{t+1}, a_{t+1} \quad (20)$$

$$\triangleq \int_{s_{t+1}, a_{t+1}} (R_t + \gamma Q(s_{t+1}, a_{t+1})) \cdot p(s_{t+1}, a_{t+1} | s_t, a_t) ds_{t+1}, a_{t+1} \quad (21)$$

$$\triangleq \int_{s_{t+1}, a_{t+1}} (R_t + \gamma Q(s_{t+1}, a_{t+1})) \cdot p(a_{t+1} | s_{t+1}) \cdot p(s_{t+1} | s_t, a_t) ds_{t+1}, a_{t+1}. \quad (22)$$

7.4 Optimal Policy

Optimal policy is a policy that maximizes $V(s_t)$. Assuming $Q(s_t, a_t)$ has an optimal policy for every policy in the future¹, it can be formulated as:

$$p^*(a_t | s_t) = \arg \max_{p(a_t | s_t)} \int_{a_t} Q^*(s_t, a_t) \cdot p(a_t | s_t) da_t \quad (23)$$

The optimal policy of $p(a_t | s_t)$ is $p^*(a_t | s_t) = \delta(a_t - a_t^*)$ where $a_t^* \triangleq \arg \max_{a_t} Q^*(s_t, a_t)$. This simply means to choose a_t that maximizes $Q^*(s_t, a_t)$. It is important to note that $a_t^* \triangleq \arg \max_{a_t} Q^*(s_t, a_t)$ is the foundation of *Greedy action* in reinforcement learning. Check your understanding about optimal policy by question 1 and 2.

8 Monte Carlo and Temporal Difference

In the previous subsection, we have seen that we had to assume $Q(s_t, a_t)$ is optimal in order to acquire a_t^* . Then how can we get Q^* ? In fact, it is not possible to acquire Q^* by solving an certain equation; rather, Q^* can be converged by using either Monte Carlo or Temporal Difference learning. Basically, we can think that there are two ways (Monte Carlo and Temporal Difference) to compute Q^* .

¹This means that $p(a_{t+1}|s_{t+1}), p(a_{t+2}|s_{t+2}), \dots$ are all optimal policies. We will represent the optimal policy by using *. For example, the optimal policy for $p(a_{t+1}|s_{t+1})$ is $p^*(a_{t+1}|s_{t+1})$

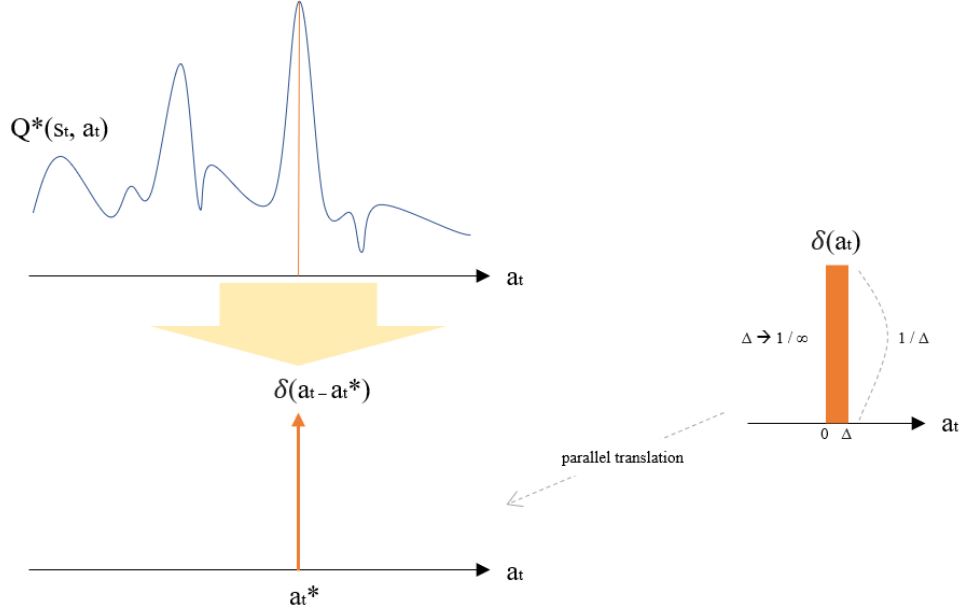


Figure 4: Visualisation of optimal policy

8.1 Monte Carlo

Monte Carlo method can be applied by using LLN (see equation ??). Retrieving $Q(s_t, a_t)$ from Action Value Function (see equation 12), we can derive the following.

$$Q(s_t, a_t) \triangleq \int_{s_{t+1}:a_{\infty}} G_t \cdot p(s_{t+1} : a_{\infty} | s_t, a_t) ds_{t+1} : a_{\infty} \quad (24)$$

$$\approx \frac{1}{N} \sum_{i=1}^N G_t^{(i)} \quad (25)$$

How the above $Q(s_t, a_t)$ get closer to $Q^*(s_t, a_t)$? It is because $p(s_{t+1} : a_{\infty} | s_t, a_t)$ will always take greedy action. By taking the greedy action for each time step, $Q(s_t, a_t)$ will also get closer to $Q^*(s_t, a_t)$. Another thing that we can observe is that $G_t^{(i)}$ can be acquired when the agent reaches to the goal.

8.2 Temporal Difference (TD)

Similar to Monte Carlo, Temporal Difference method can be applied by using LLN (see equation ??). We will retrieve the definition of $Q(s_t, a_t)$ from Bellman Equation (see equation 22).

$$Q(s_t, a_t) \triangleq \int_{s_{t+1}, a_{t+1}} (R_t + \gamma Q(s_{t+1}, a_{t+1})) \cdot p(a_{t+1} | s_{t+1}) \cdot p(s_{t+1} | s_t, a_t) ds_{t+1}, a_{t+1} \quad (26)$$

$$\approx \frac{1}{N} \sum_{i=1}^N (R_t^{(i)} + \gamma Q(s_{t+1}^{(i)}, a_{t+1}^{(i)})) \quad (27)$$

Different from Monte Carlo method, the agent does not have to reach the goal to get a sample. The agent moves only one time to acquire $Q(s_t, a_t)$. This implies that the agent considers only the next circumstance $Q(s_{t+1}, a_{t+1})$ and this is exactly what equation 27 implies. The definition of TD is not fully explained. The author of this article strongly recommends to see the next subsection as well to fully understand the meaning of TD.

8.3 MC vs TD Trade-off

Firstly, let's discuss MC. We have learned that a single example of G_t can be acquired when the agent reaches to the goal (See equation 25). The variance of the G_t will be large; since, there are many ways to reach the goal. That is, the

agent may take many steps before reaching the goal or few steps before reaching the goal. The good thing is that MC method is unbiased. This is because MC method simply chooses samples from G_t s.

On the other hand, TD method is biased; because, it samples the expectation. By looking at equation 27, we can see that $Q(s_t, a_t)$ is expectation of expectation. That is, inside $R_t^{(i)} + \gamma Q(s_{t+1}^{(i)}, a_{t+1}^{(i)})$, $Q(s_{t+1}^{(i)}, a_{t+1}^{(i)})$ is also expectation. We are sampling from expected value $Q(s_{t+1}^{(i)}, a_{t+1}^{(i)})$; hence, the sample is biased. The good thing is that TD method has small variance. This is because the agent only sees the next step.

Monte Carlo	Temporal Difference
bias ↓	bias ↑
variance ↑	variance ↓

Table 1: Comparison between MC and TD regarding bias and variance.

8.4 Incremental Monte Carlo

You might have heard about incremental Monte Carlo when studying reinforcement learning. We will talk about this and how it can be applied in TD method.

$$\overline{Q_N} \triangleq \frac{1}{N} \sum_{i=1}^N (R_t^{(i)} + \gamma Q(s_{t+1}^{(i)}, a_{t+1}^{(i)})) \quad (28)$$

$$= \frac{1}{N} (\overline{Q_{N-1}}(N-1) + R_t^{(N)} + \gamma Q(s_{t+1}^{(N)}, a_{t+1}^{(N)})) \quad (29)$$

$$= \overline{Q_{N-1}} + \frac{1}{N} (R_t^{(N)} + \gamma Q(s_{t+1}^{(N)}, a_{t+1}^{(N)}) - \overline{Q_{N-1}}) \quad (30)$$

We call this equation 30 as Incremental Monte Carlo update. So what is the meaning of this equation? Let's take a simple example to understand this. Consider $N = 10$ and we have currently 9 samples. Take a look at the following equation 31 that describes this situation. We can simply replace N to 10 in equation 30.

$$\overline{Q_{10}} = \overline{Q_9} + \frac{1}{10} (R_t^{(10)} + \gamma Q(s_{t+1}^{(10)}, a_{t+1}^{(10)}) - \overline{Q_9}) \quad (31)$$

The above implies that we can use $\overline{Q_9}$ to calculate $\overline{Q_{10}}$ which means that we can use the previous information to get the current information! We can see that the equation *incrementally* updates the $\overline{Q_N}$ and this is why it is called Incremental Monte Carlo.

Let's play with the above equation a little bit more. We set $\frac{1}{N} = \alpha$ then we can acquire the following equation.

$$\overline{Q_N} = (1 - \alpha) \cdot \overline{Q_{N-1}} + \alpha \cdot (R_t^{(N)} + \gamma Q(s_{t+1}^{(N)}, a_{t+1}^{(N)})) \quad (32)$$

We can observe that the value of α has the boundary from 0 to 1; because, N is a natural number. If α is close to 0 then we stick to the $Q(s_t, a_t)$ (previous information); whereas, if α is close to 1 then we rely on the new information $R_t^{(N)} + \gamma Q(s_{t+1}^{(N)}, a_{t+1}^{(N)})$. α basically determines how much are we going to consider the new information when updating Q . This is why α is called learning rate.

There are more things to talk before we move on to the next subsection. Firstly, have a look at the equation 30. We formally call ' $R_t^{(N)} + \gamma Q(s_{t+1}^{(N)}, a_{t+1}^{(N)}) - \overline{Q_{N-1}}$ ' as 'TD error' (more formally, it is called '1-step TD error') and ' $R_t^{(N)} + \gamma Q(s_{t+1}^{(N)}, a_{t+1}^{(N)})$ ' as 'TD target'. Secondly, have a look at the equation 32. If we use equation 32 to update Q , then we call this SARSA algorithm and this could be formally expressed as the following:

$$Q(s_t, a_t) \leftarrow (1 - \alpha) \cdot Q(s_t, a_t) + \alpha \cdot (R_t + \gamma Q(s_{t+1}, a_{t+1})) \quad (33)$$

If we choose the best action a_{t+1} among a_{t+1} s, then we call it Q-Learning and it could be formally expressed as the following:

$$Q(s_t, a_t) \leftarrow (1 - \alpha) \cdot Q(s_t, a_t) + \alpha \cdot (R_t + \gamma \max_{a_{t+1}} Q(s_{t+1}, a_{t+1})) \quad (34)$$

9 n-step Temporal Difference

10 Model-free vs Model-based

In Markov Decision Process, there are two properties: transition function and reward function. The transition function is a function that, given a state and an action, outputs a probability of moving to a new state. The reward function is a function that outputs a reward, given a state and an action. Both the transition function and the reward function together, it is called as ‘model’ of environment. In general, it is known that if we have the transition and reward functions (if the model is given), we can use policy iteration or value iteration (dynamic programming) to compute an optimal policy. An model-based algorithm uses the transition and reward functions to estimate the optimal policy. In summary, a MDP model is the problem and the optimal policy is the solution to the problem.

However, we do not always have the model which means that we do not always have the transition and reward functions. That is, we cannot obtain the policy from MDP when the MDP is unknown. In this case, the agent needs to interact with environment and estimate the optimal policy. Over the time steps, the agent understands the environment by taking actions and getting rewards. This is so called as ‘trial and error’ approach. A model-free algorithm does not use the transition and reward functions to estimate the optimal policy.

There is a good way to distinguish between model-based and model-free algorithms. We can see if the algorithm uses the transition or reward functions. For example, Q-Learning and SARSA algorithms are model-free because it does not use any probabilities (transition or reward functions) defined by the MDP. On the other hand, the policy improvement algorithm uses probabilities defined by the MDP (See equation 35). Therefore, policy iteration which uses the policy improvement algorithm is a model-based.

$$Q(s_t, a_t) \leftarrow \sum_{s_{t+1} \in \mathcal{S}, R_t \in \mathcal{R}} p(s_{t+1}, R_t | s_t, a_t) (R_t + \gamma V(s_{t+1})) \quad (35)$$

11 Q-Learning vs SARSA

We will now see more about Q-Learning and SARSA. Q-Learning algorithm is one of the most renowned reinforcement learning algorithm. It is very crucial to fully understand this algorithm; since, there are many algorithms based on Q-Learning. Let’s have a look at some basic topics before we move on to Q-Learning and SARSA.

11.1 Exploitation and Exploration

Reinforcement learning is all about finding the optimal policy and getting the optimal reward from the given environment. To do so, we should use exploitation and exploration algorithm. Let’s see why this is important and how this could be implemented. Let’s have a look at Figure 5.

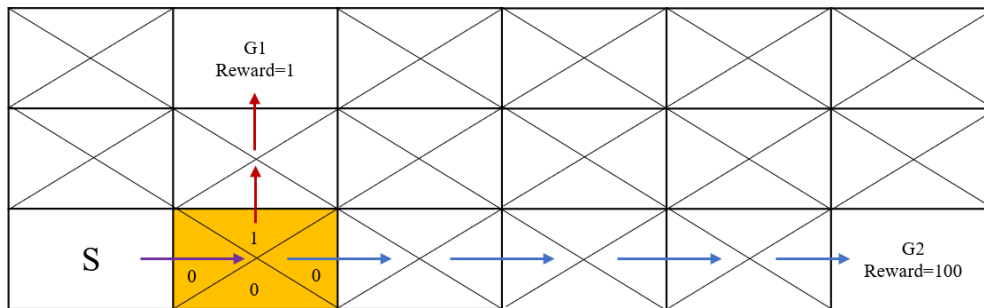


Figure 5: Red path: The first path that the agent found. Blue path: Another path that the agent can reach. Purple path: shared path

Imagine an agent starts from state S and found the path to $G1$ that rewards 1 to the agent. Assume the agent is using Q-Learning update rule. As you might have noticed, it is more likely to find $G1$ beforehand $G2$ because $G1$ is closer to S . This means that, without any exploration, the agent will always exploit the founded path (red path) even though there is a better state that rewards 100 (blue path). Here we can utilize exploitation and exploration algorithms.

11.1.1 ϵ -greedy

The first algorithm is ϵ -greedy (called epsilon greedy). ϵ -greedy enables the agent to find the better path by randomly moving on to the next state with probability ϵ . If you look at Figure 5, we can see that $Q(\text{orange_state}, \text{up}) = 1$ and $Q(\text{orange_state}, -\text{up}) = 0$. Without ϵ -greedy, the agent will always move to the upper state. However, the agent can explore to another state (e.g. *right, down, left*) by utilising ϵ -greedy. In this manner, the agent can find the blue path which gives more rewards.

ϵ -greedy Action Selection

```
Function SELECT-ACTION( $Q, \epsilon$ ):  
   $n \leftarrow$  uniform random number between 0 and 1;  
  if  $n < \epsilon$  then  
     $A \leftarrow$  random action from the actions;  
  else  
     $A \leftarrow \arg \max_a Q(s, a)$ ;  
  end if  
  return selected action  $A$ ;  
end
```

Algorithm 1: ϵ -greedy

In fact, decaying ϵ -greedy is more general algorithm. The idea is very simple. The agent slowly decreases the degree of exploration for every episode. The agent does not have enough information in early episodes leading to explore more in early stage. After many episodes, the agent decreases the exploration rate to exploit the information.

Decaying ϵ -greedy Action Selection

```
for each episode  $i$  do:  
  /* do something */;  
   $e \leftarrow \epsilon / (i + 1)$ ;  
   $a \leftarrow \text{SELECT-ACTION}(Q, e)$ ;  
  /* do something */;  
end
```

Algorithm 2: Decaying ϵ -greedy

11.1.2 Adding noise

The second algorithm is to add random noises to Q . To simplify the algorithm, let's assume there are only 4 actions that could be selected in a state.

Action Selection with random noise

```
Function SELECT-ACTION( $Q, \text{noise}$ ):  
   $n_1, n_2, n_3, n_4 \leftarrow$  random noise (noise value depends on a model);  
   $Q(s, a_i) \leftarrow Q(s, a_i) + n_i$  ;  
  return  $Q(s, a_i)$ ;  
end
```

Algorithm 3: Random noise

ϵ -greedy algorithm will choose any action when n is smaller than ϵ . However, unlike ϵ -greedy algorithm, random noise algorithm helps the agent to avoid choosing the worst action but gives a chance to choose the second or third best action in a state (See Figure 6).

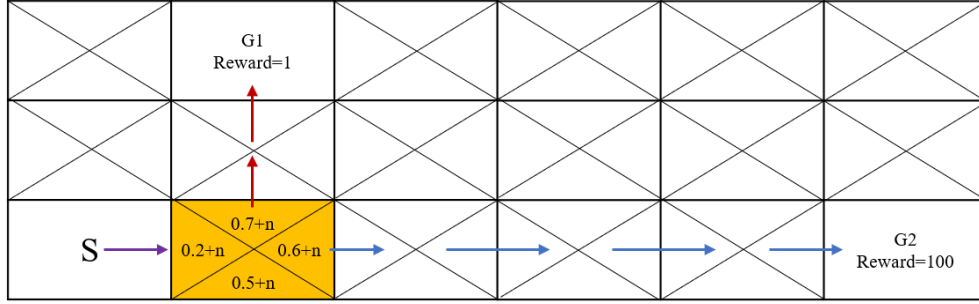


Figure 6: n denotes noise and subscripts of n is removed for simplification. Red path: The first path that the agent found. Blue path: Another path that the agent can reach. Purple path: shared path

11.2 Revisiting Discount Factor (γ)

Let's assume that an agent resides in 3 X 3 grid and the agent updates the grid by using Q-learning (See equation 34). There are many ways to reach the goal from starting state. Figure 7 shows two possible examples of the ways. Which one do you think it is more efficient? Indeed, the middle one looks more efficient than the other. The agent knows that moving to *right* is better than moving to *up* in *orange_state* because the $Q(\text{orange_state}, \text{right}) = 0.9$ is greater than $Q(\text{orange_state}, \text{up}) = 0.73$. Therefore, we can think the discount factor helps the agent to find the optimal path to the goal.

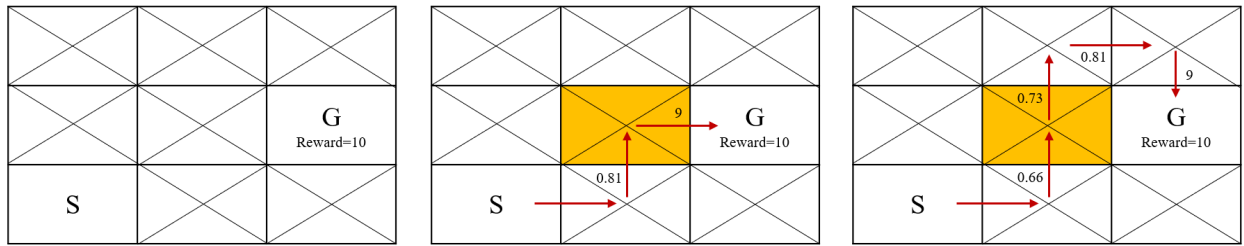


Figure 7: Visualising the effect of discount factor γ . S: starting state. G: Goal. Left board: Initialised Board. Middle board: The agent takes 3 steps to reach the goal. Right board: The agent takes 5 steps to reach the goal.

11.3 Q-Learning

Q-learning (off-policy TD control)

Algorithm hyper-parameters: learning rate (or step size) $\alpha \in (0, 1]$, $\varepsilon > 0$;

Initialize $Q(s, a)$, for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$, arbitrarily, and $Q(\text{terminal_state}, \cdot) = 0$;

foreach *episode* **do**

 Initialize all states (place an agent at the starting state);

foreach *step* **do**

 Choose a_t from s_t using policy derived from Q (e.g., ε -greedy);

 Take action a_t , observe R_t , s_{t+1} ;

$Q(s_t, a_t) \leftarrow (1 - \alpha)Q(s_t, a_t) + \alpha(R_t + \gamma \max_{a_{t+1}} Q(s_{t+1}, a_{t+1}))$;

$s_t \leftarrow s_{t+1}$;

 (break when s_t is terminal)

end foreach

end foreach

Algorithm 4: Q-learning

11.4 SARSA

SARSA (on-policy TD control)

Algorithm hyper-parameters: learning rate (or step size) $\alpha \in (0, 1]$, $\varepsilon > 0$;

Initialize $Q(s, a)$, for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$, arbitrarily, and $Q(\text{terminal_state}, \cdot) = 0$;

foreach *episode* **do**

 Initialize all states (place an agent at the starting state);

 Choose a_t from s_t using policy derived from Q (e.g., ε -greedy);

foreach *step* **do**

 Take action a_t , observe R_t, s_{t+1} ;

 Choose a_{t+1} from s_{t+1} using policy derived from Q (e.g., ε -greedy);

$Q(s_t, a_t) \leftarrow (1 - \alpha)Q(s_t, a_t) + \alpha(R_t + \gamma Q(s_{t+1}, a_{t+1}))$;

$s_t \leftarrow s_{t+1}$;

$a_t \leftarrow a_{t+1}$;

 (break when s_t is terminal)

end foreach

end foreach

Algorithm 5: SARSA

12 Deep Q-Learning

In this section, we will cover several recent papers about deep Q-learning. We firstly need to know that why there is a word 'deep' in front of Q-Learning. Take a look at the following Figure ??.

12.1 DQN (NIPS 2013)

12.2 DQN (Nature 2015)

12.3 Double DQN (DDQN)

12.4 Dueling DQN

12.5 Prioritized Experience Replay (PER)

12.6 A Distributional Perspective on Reinforcement Learning

13 Policy Based Algorithms

Policy based algorithms are different from value based algorithms (i.e. Q-learning and SARSA). One of the critical problem of value based approach to solve reinforcement learning problem is that it could be only used in scenarios where actions are discrete. For example, let's assume that we would like to make the robot that moves its legs to move forward. The action space of the robot should be continuous rather than discrete, because the action space will be defined by an angle (See Figure 8). If the action is discrete, we should assign a node for each action, which means that $node_0$ corresponds to $\theta = 0$, $node_1$ corresponds to $\theta = 0.001$, and so on. In this way, it will be impossible for a DQN model to properly adjust its parameters to find the best action for each state.

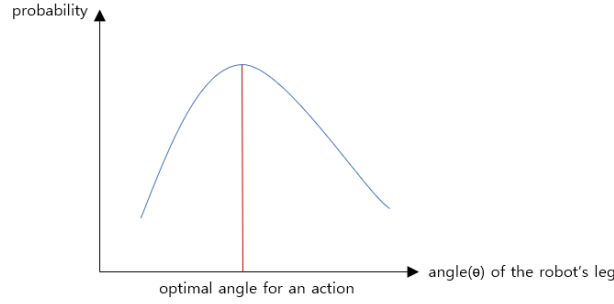


Figure 8: The action space of the robot is continuous.

In addition, when the action is continuous, Q function becomes complicated (i.e. Q function is non-linear). Finding the action that maximizes Q: $\arg \max_{a_t} Q(a_t | s_t)$ turns into a new optimization problem as we are going to use gradient descent method (See Figure 9). We need to solve an optimization problem for every step and this is not trivial.

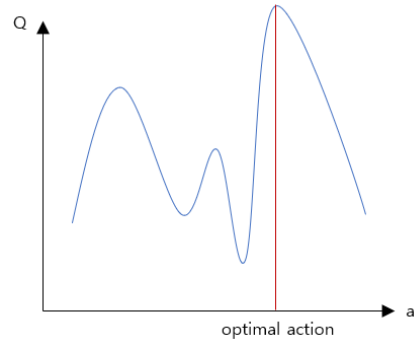


Figure 9: Finding the optimal action becomes an optimization problem.

Another advantage of policy based algorithms over value based algorithms is that the policy based algorithms can find a stochastic policy. This means that the policy can be represented by a probability distribution. We can parameterize(θ) the policy like Figure 10 and samples from the distribution to get the policy.

This method is very useful in the following scenario (See Figure 11). Let's assume the state can be represented by the following $s = [up, right, down, left]$. If there is a wall on that side then it is 1 otherwise 0. For instance, if the agent is in the grey area then $s = [1, 0, 1, 0]$. We call this *partially observable state* which means that the agent cannot distinguish the left grey state and right grey state, but we know that the left and right are different. The agent will think that the two states are the exactly same state. We will assume that the agent learns this game by using DQN and the agent is now at the grey state. There are two actions that the agent can perform: left or right. If $Q(grey, left) > Q(grey, right)$ then the agent moves to left (case 1) and if $Q(grey, left) < Q(grey, right)$ then the agent moves to right (case 2). When the agent is in the left grey state and case 1, the agent gets stuck, wandering between s_0 and s_1 . When the agent is in the right grey state and case 1, the agent can reaches to the goal. The exact same problem arises for the second case (case 2) as well. Then, what will be the best policy for

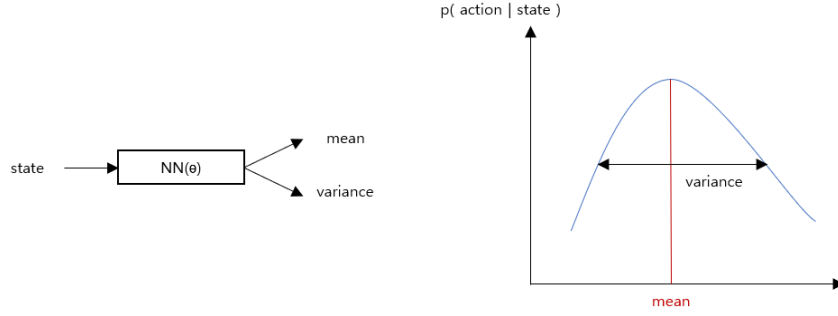


Figure 10: Input: state, Output: mean and variance of a policy

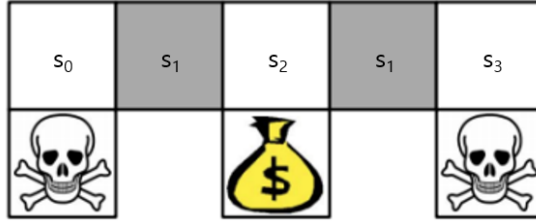


Figure 11: Understanding stochastic policy by using simple example. The agent cannot differentiate between the left and right grey state. This image is from Stanford CS234.

state s_1 ? The best policy for state s_1 is $p(left|s_1) = p(right|s_1) = \frac{1}{2}$. This can be realized by using a stochastic policy.

We have learned that policy based algorithm is better than the other in terms of two aspects: continuous action and stochastic policy. What how can we define our new objective function of policy based algorithm? In reinforcement learning, our goal is to always maximize rewards/returns. More precisely, it is to maximize *expected* rewards/returns. In policy based, maximizing the *expected* rewards/returns from the beginning is our goal. Let's retrieve the definition of return from equation ??.

$$G_0 = R_0 + \gamma R_1 + \gamma^2 R_2 + \dots \quad (36)$$

A policy based algorithm tries to maximize the expectation of G_0 , namely $E[G_0]$. We will set $E[G_0]$ as our objective function and our goal is to maximize this.

$$J \triangleq E[G_0] \quad (37)$$

$$= \int_{s_0: a_\infty} G_0 \cdot p(s_0, a_0, s_1, a_1 \dots) ds_0 : a_\infty \quad (38)$$

$$(39)$$

by setting $\tau = s_0, a_0, s_1, a_1 \dots$ we get the following (τ represents a trajectory):

$$J = \int_{\tau} G_0 \cdot p(\tau) d\tau \quad (40)$$

To maximize the expectation of G_0 , we should find the optimal policy for each state. Then, where is the policy in equation 40? It is inside $p(\tau)$. Inside $p(\tau)$, we have $p_\theta(a_0|s_0), p_\theta(a_1|s_1), \dots$. We can rewrite the objective function with parameter θ . See Figure 10 to see how the policy can be parameterized.

$$J_\theta = \int_{\tau} G_0 \cdot p_\theta(\tau) d\tau \quad (41)$$

The objective function is now defined and we have parameterized the function by using θ . We can now differentiate the function by θ and use gradient ascent method to maximize the objective function.

$$\nabla_\theta J_\theta = \nabla_\theta \int_{\tau} G_0 \cdot p_\theta(\tau) d\tau \quad (42)$$

$$\theta \leftarrow \theta + \alpha \cdot \nabla_\theta J_\theta \quad (43)$$

where

$$\nabla_{\theta} J_{\theta} \triangleq \frac{\partial J_{\theta}}{\partial \theta} \quad (44)$$

We call $\nabla_{\theta} J_{\theta}$ **policy gradient**. We will simplify equation 42. After simplification, we will be able to redefine $\nabla_{\theta} J_{\theta}$ using the policy $p(a|s)$. Calculating summation after differentiation (equation 45) and calculating differentiation after summation 42 is the same.

$$\nabla_{\theta} J_{\theta} = \int_{\tau} G_0 \cdot \nabla_{\theta} p_{\theta}(\tau) d\tau \quad (45)$$

Equation 45 is no longer expectation. Thus, we apply a trick to make the equation into an expectation format.

$$\nabla_{\theta} J_{\theta} = \int_{\tau} G_0 \cdot (\nabla_{\theta} \ln p_{\theta}(\tau)) \cdot p_{\theta}(\tau) d\tau \quad (46)$$

We can now make gradient with sample means. We will try to simplify $G_0 \cdot (\nabla_{\theta} \ln p_{\theta}(\tau))$. Firstly, G_0 can be expressed by the following:

$$G_0 = R_0 + \gamma R_1 + \gamma^2 R_2 + \dots \quad (47)$$

Secondly, we can rewrite $p_{\theta}(\tau)$ by the following:

$$p_{\theta}(\tau) = p_{\theta}(s_0, a_0, s_1, a_1, s_2, a_2, \dots) \quad (48)$$

$$= p(s_0) \cdot p_{\theta}(a_0, s_1, a_1, \dots | s_0) \quad (49)$$

$$= p(s_0) \cdot p_{\theta}(a_0 | s_0) \cdot p_{\theta}(s_1, a_1, \dots | s_0, a_0) \quad (50)$$

$$= p(s_0) \cdot p_{\theta}(a_0 | s_0) \cdot p(s_1 | s_0, a_0) \cdot p_{\theta}(a_1, s_2, a_2, \dots | s_0, a_0, s_1) \quad (51)$$

$$= p(s_0) \cdot p_{\theta}(a_0 | s_0) \cdot p(s_1 | s_0, a_0) \cdot p_{\theta}(a_1 | s_0, a_0, s_1) \cdot p_{\theta}(s_2, a_2, \dots | s_0, a_0, s_1, a_1) \quad (52)$$

$$= p(s_0) \cdot p_{\theta}(a_0 | s_0) \cdot p(s_1 | s_0, a_0) \cdot p_{\theta}(a_1 | s_0, a_0, s_1) \cdot p(s_2 | s_0, a_0, s_1, a_1) \cdot p_{\theta}(a_2, s_3, a_3, \dots | s_0, a_0, s_1, a_1, s_2) \quad (53)$$

$$= p(s_0) \cdot p_{\theta}(a_0 | s_0) \cdot p(s_1 | s_0, a_0) \cdot p_{\theta}(a_1 | s_1) \cdot p(s_2 | s_1, a_1) \cdot p_{\theta}(a_2, s_3, a_3, \dots | s_2) \quad (54)$$

$$= \dots \quad (55)$$

It is important to note that the parameter θ is only applied to the policies. It can be observed that there is no θ in $p(s_0)$ and other transition probabilities, namely $p(s_{t+1} | s_t, a_t)$. In logarithms, multiplication can be expressed by summation. Also, $\ln p_{\theta}(\tau)$ is differentiated by θ . Therefore, by calculating $\nabla_{\theta} \ln p_{\theta}(\tau)$, we get the following:

$$\nabla_{\theta} \ln p_{\theta}(\tau) = \nabla_{\theta} \sum_{t=0}^{\infty} \ln p_{\theta}(a_t | s_t) \quad (56)$$

We can rewrite $\nabla_{\theta} J_{\theta}$ by the following:

$$\nabla_{\theta} J_{\theta} = \int_{\tau} G_0 \cdot (\nabla_{\theta} \ln p_{\theta}(\tau)) \cdot p_{\theta}(\tau) d\tau \quad (57)$$

$$= \int_{\tau} (R_0 + \gamma R_1 + \gamma^2 R_2 + \dots) \cdot (\nabla_{\theta} \sum_{t=0}^{\infty} \ln p_{\theta}(a_t | s_t)) \cdot p_{\theta}(\tau) d\tau \quad (58)$$

$$= \int_{\tau} \sum_{t=0}^{\infty} (\nabla_{\theta} \ln p_{\theta}(a_t | s_t) \cdot (\sum_{k=t}^{\infty} \gamma^k \cdot R_k)) \cdot p_{\theta}(\tau) d\tau \quad (59)$$

$$= \int_{\tau} \sum_{t=0}^{\infty} (\nabla_{\theta} \ln p_{\theta}(a_t | s_t) \cdot (\sum_{k=t}^{\infty} \gamma^t \cdot \gamma^{k-t} \cdot R_k)) \cdot p_{\theta}(\tau) d\tau \quad (60)$$

$$= \int_{\tau} \sum_{t=0}^{\infty} (\nabla_{\theta} \ln p_{\theta}(a_t | s_t) \cdot (\gamma^t \cdot G_t)) \cdot p_{\theta}(\tau) d\tau \quad (61)$$

When t gets larger, γ^t becomes very small. This implies that we apply very small weight to the future reward, therefore the agent cannot learn properly. After removing γ^t , we can acquire the simplified version of equation 42. You might wonder how the equation 58 could be simplified into 59. See 14.2 for more details.

$$\nabla_{\theta} J_{\theta} \approx \int_{\tau} \sum_{t=0}^{\infty} (\nabla_{\theta} \ln p_{\theta}(a_t | s_t) \cdot G_t) \cdot p_{\theta}(\tau) d\tau \quad (62)$$

13.1 REINFORCE

Deriving REINFORCE algorithm begins from the equation 62. We can approximate the equation 62 by using sampling.

$$\nabla_{\theta} J_{\theta} \approx \frac{1}{N} \sum_{i=1}^N \left(\sum_{t=0}^{\infty} (\nabla_{\theta} \ln p_{\theta}(a_t^{(i)} | s_t^{(i)}) \cdot G_t^{(i)}) \right) \quad (63)$$

where

$$\sum_{t=0}^{\infty} (\nabla_{\theta} \ln p_{\theta}(a_t | s_t) \cdot G_t) \sim p_{\theta}(\tau) \quad (64)$$

The time step t starts from 0 to ∞ for each sample. Basically, we can get a single sample when the agent goes into the terminal state (i.e. when the episode finishes). Hence, there will be too much computation if we want to sample many samples (i.e. large N). To reduce the computation, we can set $N = 1$. Rewriting the objective function, we get the following:

$$\nabla_{\theta} J_{\theta} \approx \sum_{t=0}^{\infty} (\nabla_{\theta} \ln p_{\theta}(a_t | s_t) \cdot G_t) \quad (65)$$

REINFORCE algorithm

Algorithm parameters: policy parameter θ ;
 Algorithm hyper-parameters: maximum episode T and learning rate α ;
 Initialize policy parameter θ , choose maximum episode T , and learning rate α ;
foreach *episode* **do**
 Generate a trajectory $\tau : s_0, a_0, R_0, \dots$, following $p_{\theta}(a_t | s_t)$;
 Calculate G_t using R_t where $t = 0, 1, \dots, T - 1$;
 foreach *step* **do**
 $\theta \leftarrow \theta + \alpha \cdot \sum_{t=0}^{\infty} (\nabla_{\theta} \ln p_{\theta}(a_t | s_t) \cdot G_t)$;
 end foreach
 (Break when *episode* is equal to T)
end foreach

Algorithm 6: REINFORCE algorithm

REINFORCE algorithm is unbiased but high variance. It is unbiased, because we have used sampling method. However, its variance is very high, as a single trajectory that we have sampled would be highly different from the other trajectories that we will sample afterwards. The algorithm would take very long time to converge because of high variance.

13.2 Actor Critic (AC)

We will start from the equation 62 to induce Actor-Critic algorithm.

$$\nabla_{\theta} J_{\theta} \approx \int_{\tau} \sum_{t=0}^{\infty} (\nabla_{\theta} \ln p_{\theta}(a_t | s_t) \cdot G_t) \cdot p_{\theta}(\tau) d\tau \quad (66)$$

$$= \int_{\tau} \sum_{t=0}^{\infty} (\nabla_{\theta} \ln p_{\theta}(a_t | s_t) \cdot G_t) \cdot p_{\theta}(s_{t+1}, a_{t+1}, \dots | s_0, a_0, \dots, s_t, a_t) \cdot p_{\theta}(s_0, a_0, \dots, s_t, a_t) d\tau \quad (67)$$

$$= \sum_{t=0}^{\infty} \int_{\tau} (\nabla_{\theta} \ln p_{\theta}(a_t | s_t) \cdot G_t) \cdot p_{\theta}(s_{t+1}, a_{t+1}, \dots | s_0, a_0, \dots, s_t, a_t) \cdot p_{\theta}(s_0, a_0, \dots, s_t, a_t) d\tau \quad (68)$$

$$= \sum_{t=0}^{\infty} \int_{\tau} (\nabla_{\theta} \ln p_{\theta}(a_t | s_t) \cdot G_t) \cdot p_{\theta}(s_{t+1}, a_{t+1}, \dots | s_t, a_t) \cdot p_{\theta}(s_0, a_0, \dots, s_t, a_t) d\tau \quad (69)$$

$$= \sum_{t=0}^{\infty} \int_{s_0, a_0, \dots, s_t, a_t} \nabla_{\theta} \ln p_{\theta}(a_t | s_t) \quad (70)$$

$$\int_{s_{t+1}, a_{t+1}, \dots} G_t \cdot p_{\theta}(s_{t+1}, a_{t+1}, \dots | s_t, a_t) ds_{t+1}, a_{t+1}, \dots p_{\theta}(s_0, a_0, \dots, s_t, a_t) ds_0, da_0, \dots, s_t, a_t \quad (71)$$

Considering the equation 12, the above could be simplified by the following. The definition of Q is used.

$$\nabla_{\theta} J_{\theta} \approx \sum_{t=0}^{\infty} \int_{s_0, a_0, \dots, s_t, a_t} \nabla_{\theta} \ln p_{\theta}(a_t | s_t) \cdot Q(s_t, a_t) \cdot p_{\theta}(s_0, a_0, \dots, s_t, a_t) ds_0, da_0, \dots, s_t, a_t \quad (72)$$

After applying marginalization to p_{θ} , we get the following:

$$\nabla_{\theta} J_{\theta} \approx \sum_{t=0}^{\infty} \int_{s_t, a_t} \nabla_{\theta} \ln p_{\theta}(a_t | s_t) \cdot Q(s_t, a_t) \cdot p_{\theta}(s_t, a_t) ds_t, a_t \quad (73)$$

By solving the equation, we have replaced G into Q . Considering the fact that we were able to calculate G when the episode terminates, we no longer need to wait for the episode to be terminated by using Q . In Actor-Critic algorithm, Q is parameterized by w and it gets updated similar to SARSA algorithm.

$$\nabla_{\theta} J_{\theta} \approx \sum_{t=0}^{\infty} \int_{s_t, a_t} \nabla_{\theta} \ln p_{\theta}(a_t | s_t) \cdot Q_w(s_t, a_t) \cdot p_{\theta}(s_t, a_t) ds_t, a_t \quad (74)$$

where Q_w gets updated by the following TD-error:

$$\|(R_t + \gamma \cdot Q_w(s_{t+1}, a_{t+1})) - Q_w(s_t, a_t)\|_2^2 \quad (75)$$

An actor updates $p_{\theta}(a_t | s_t)$ and a critic updates Q_w . Unlike REINFORCE algorithm, Actor-Critic algorithm updates θ and w for every step. This is possible since Q can be updated once s_t and a_t are acquired by the policy. This implies that we do not take $\sum_{t=0}^{\infty}$ into account when running the algorithm. Instead of waiting for the episode to be finished, 1-step TD is used. Summing up all the details, we can write the following Actor-Critic algorithm.

Actor-Critic algorithm

Algorithm parameters: actor(policy) parameter θ and critic parameter w ;
Algorithm hyper-parameters: maximum step T , discount factor γ , learning rate for actor α , and critic β ;

Initialize θ and w . Choose maximum step T , discount factor γ , learning rate α , and β ;

repeat

 Actor update: $\theta \leftarrow \theta + \alpha \cdot Q_w(s_t, a_t) \cdot \nabla_{\theta} \ln p_{\theta}(a_t | s_t)$;

 Critic update: $w \leftarrow w + \beta \cdot (R_t + \gamma Q_w(s_{t+1}, a_{t+1}) - Q_w(s_t, a_t)) \cdot \nabla_w Q_w(s_t, a_t)$

until we find satisfactory parameters: θ and w ;

Algorithm 7: Actor-Critic algorithm

The algorithm is biased and low variance compared to REINFORCE algorithm. It is biased, because we ignored $\sum_{t=0}^{\infty}$ and used 1-step TD error. In addition, we have parameterized Q with w . On the contrary, it is low variance, since we have replaced G with Q . In fact, it was mentioned in subsection 8.3 that (1-step) TD method is biased and low variance.

13.3 Advantage Actor Critic (A2C)

Previously, we have covered that Actor Critic algorithm helps to reduce the variance compared to REINFORCE algorithm. Similarly, A2C algorithm aims to have less variance than AC algorithm. We will start from the following equation to derive A2C.

$$\nabla_{\theta} J_{\theta} \approx \sum_{t=0}^{\infty} \int_{s_t, a_t} \nabla_{\theta} \ln p_{\theta}(a_t | s_t) \cdot Q_w(s_t, a_t) \cdot p_{\theta}(s_t, a_t) ds_t, a_t \quad (76)$$

The question is, if we plug $b(s_t)$ instead of $Q(s_t, a_t)$, will $b(s_t)$ affect the integration? Let's see whether $b(s_t)$ affect the integration or not. Plugging $b(s_t)$ instead of $Q(s_t, a_t)$,

$$\nabla_{\theta} J_{\theta} \approx \sum_{t=0}^{\infty} \int_{s_t, a_t} \nabla_{\theta} \ln p_{\theta}(a_t | s_t) \cdot b(s_t) \cdot p_{\theta}(s_t, a_t) ds_t, a_t \quad (77)$$

$$= \sum_{t=0}^{\infty} \int_{s_t, a_t} \frac{\nabla_{\theta} p_{\theta}(a_t | s_t)}{p_{\theta}(a_t | s_t)} \cdot b(s_t) \cdot p_{\theta}(a_t | s_t) \cdot p(s_t) ds_t, a_t \quad (78)$$

$$= \sum_{t=0}^{\infty} \int_{s_t} \int_{a_t} \nabla_{\theta} p_{\theta}(a_t | s_t) da_t b(s_t) \cdot p(s_t) ds_t \quad (79)$$

$$(80)$$

Here, we can observe the following,

$$\int_{a_t} \nabla_{\theta} p_{\theta}(a_t | s_t) da_t \quad (81)$$

$$\nabla_{\theta} \int_{a_t} p_{\theta}(a_t | s_t) da_t = 0 \quad (82)$$

The above proves that if we replace $Q(s_t, a_t)$ with any function that is not related with the action, then the function (specifically, $b(s_t)$ and it is called *baseline*) does not affect $\nabla_{\theta} J_{\theta}$. Instead of using $b(s_t)$, we will use $V(s_t)$ from now on. In addition, we have proved that the function without the action does not affect the integration. Hence, we will substitute $Q(s_t, a_t) - V(s_t)$ for $Q(s_t, a_t)$. It is known that the sample variance reduces when using the above substitution. The substitution is not the optimal way to reduce the variance. However, it is the most commonly used way to reduce the variance. We call $Q(s_t, a_t) - V(s_t)$ as *advantage*. Solving the equation with $Q(s_t, a_t) - V(s_t)$,

$$\nabla_{\theta} J_{\theta} \approx \sum_{t=0}^{\infty} \int_{s_t, a_t} \nabla_{\theta} \ln p_{\theta}(a_t | s_t) \cdot (Q_w(s_t, a_t) - V(s_t)) \cdot p_{\theta}(s_t, a_t) ds_t, a_t \quad (83)$$

$$= \sum_{t=0}^{\infty} E_{s_t, a_t} [\nabla_{\theta} \ln p_{\theta}(a_t | s_t) \cdot (Q - V)] \quad (84)$$

The problem of the above equation is that we have too many parameters. We have already parameterized Q and p with w and θ , respectively. But, $V(s_t)$ has been introduced, hence we again need to parameterize it. In order to overcome the issue, we should use the equation 18. This allows us to use only two parameters. Rewriting the gradient,

$$\nabla_{\theta} J_{\theta} \approx \sum_{t=0}^{\infty} E_{s_t, a_t} [\nabla_{\theta} \ln p_{\theta}(a_t | s_t) \cdot (Q - V)] \quad (85)$$

$$= \sum_{t=0}^{\infty} E_{s_t, a_t} [\nabla_{\theta} \ln p_{\theta}(a_t | s_t) \cdot E_{s_{t+1}} [R_t + \gamma \cdot V(s_{t+1}) - V(s_t) | s_t, a_t]] \quad (86)$$

As the equation is formulated with only V and p , we can parameterize them with w and θ , respectively. We will expand the above expectation to the integration format, and we get the following.

$$\nabla_{\theta} J_{\theta} \approx \sum_{t=0}^{\infty} \int_{s_t, a_t} \nabla_{\theta} \ln p_{\theta}(a_t | s_t) \int_{s_{t+1}} (R_t + \gamma V(s_{t+1}) - V(s_t)) \cdot p(s_{t+1} | s_t, a_t) ds_{t+1} p_{\theta}(s_t, a_t) ds_t, a_t \quad (87)$$

$$= \sum_{t=0}^{\infty} \int_{s_t, a_t, s_{t+1}} \nabla_{\theta} \ln p_{\theta}(a_t | s_t) \cdot (R_t + \gamma V(s_{t+1}) - V(s_t)) \cdot p(s_{t+1} | s_t, a_t) \cdot p(s_t) \cdot p_{\theta}(a_t | s_t) ds_t, a_t, s_{t+1} \quad (88)$$

The sample for the above equation is: $\ln p_\theta(a_t|s_t) \cdot (R_t + \gamma V(s_{t+1}) - V(s_t))$, and the sample follows $p(s_{t+1}|s_t, a_t)$, $p(s_t)$, and $p_\theta(a_t|s_t)$. This implies that once θ gets updated by the sample that we have sampled, we should sample from the updated distribution for the next time step. For example, after we sample N times from the distribution which is constructed with θ_t and update θ ($\theta_{t+1} \leftarrow \theta_t$), we should sample from the distribution created with θ_{t+1} for the next time step to update θ ($\theta_{t+2} \leftarrow \theta_{t+1}$). We can visualize this by the following figure 12.

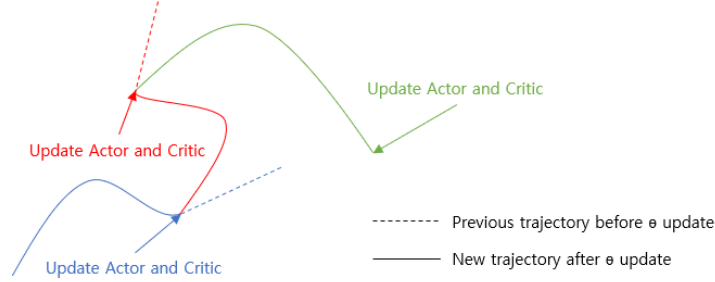


Figure 12: Visualisation of A2C algorithm

At this point, you might wonder how does $p(s_t)$ can be calculated. In fact, $p(s_t)$ can be rewritten as $p(s_t|s_{t-1}, a_{t-1})$. Summing up all the details that we have seen, we get the following equation:

$$\nabla_\theta J_\theta \approx \sum_{t=0}^{\infty} \int_{s_t, a_t, s_{t+1}} \nabla_\theta \ln p_\theta(a_t|s_t) \cdot (R_t + \gamma V(s_{t+1}) - V(s_t)) \cdot p(s_{t+1}|s_t, a_t) \cdot p_\theta(a_t|s_t) \cdot p(s_t|s_{t-1}, a_{t-1}) ds_t, a_t, s_{t+1} \quad (89)$$

It is strongly recommended to refer Figure 13 with Algorithm 8. As you might have noticed, A2C algorithm uses N samples to update the parameters θ and w unlike to AC algorithm which uses a single sample. AC algorithm did not take $\sum_{t=0}^{\infty}$ into account. However, A2C algorithm considers $\sum_{t=0}^{\infty}$ and use N samples to update parameters. This is possible because we have used $Q - V$ instead of Q . By subtracting V , it was able to reduce variance; therefore, now we can use more samples. $Q - V$ reduces the variance and using more than one sample increases the variance. They cancel each other in terms of variance.

Advantage Actor-Critic algorithm

Algorithm parameters: actor(policy) parameter θ and critic parameter w ;

Algorithm hyper-parameters: maximum step T , discount factor γ , sample size N , learning rate for actor α , and critic β ;

Initialize θ and w . Choose maximum step T , discount factor γ , sample size N , learning rate α , and β ;

repeat

 Collect N samples (single sample: s_t, a_t, R_t, s_{t+1});

 Actor update: $\theta \leftarrow \theta + \alpha \cdot \nabla_\theta \sum_{i=t-N+1}^t \ln p_\theta(a_i|s_i) \cdot (R_i + \gamma V_w(s_{i+1}) - V_w(s_i))$;

 Critic update: $w \leftarrow w - \beta \cdot \nabla_w \sum_{i=t-N+1}^t (R_i + \gamma V_w(s_{i+1}) - V_w(s_i))^2$;

 Clear the batch (Remove N samples)

until we find satisfactory parameters: θ and w ;

Algorithm 8: Advantage Actor-Critic algorithm

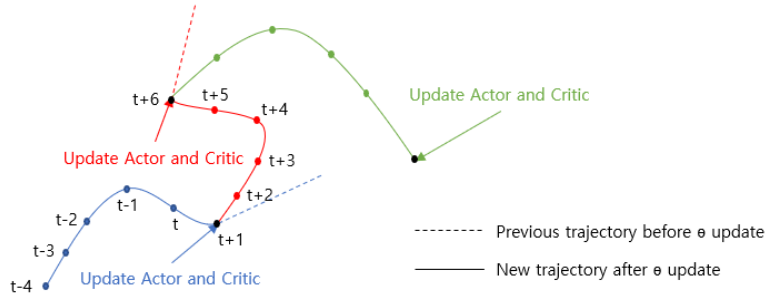


Figure 13: Visualisation of A2C algorithm when $N = 5$. Critic gets updated with $N = 5$ TD-errors

13.4 Asynchronous Advantage Actor Critic (A3C)

There three important aspects in A3C which are (1) correlations between batches, (2) n-step TD, and (3) Entropy. Let's find out what these are.

13.4.1 Correlations between batches

In A2C algorithm, we used batches to update both actor and critic. For example, if you see Figure 13, you can observe that we have used 5 samples to update them ($N = 5$). As we update them with consecutive experiences, it generates correlation between batches. This causes a problem because when we update a neural network, samples (or batches) should not be correlated (i.e. samples should satisfy i.i.d. property unless it is full gradient. Using i.i.d. samples for updating gradient could be considered as an unbiased estimate of full gradient.). A3C algorithm tries to alleviate the correlations between batches by introducing asynchronous (or multiple) agents. The agents does not know the existence of other agents and generates samples independently. Although the agents share the parameter (θ in $p_\theta(a_t|s_t)$), their trajectories are different from each other because they sample an action from the distribution $p_\theta(a_t|s_t)$. We can visualize the above by the following:



Figure 14: Asynchronous agents in A3C algorithm. Different agent (agent ①, ②, and ③) can have different trajectory. It is possible to have different batch size for each agent.

We call each agent as *worker*. Each worker gives either samples or calculated gradient to the center, and the center gives updated parameters to each worker back (See Figure 14). Each worker produces samples and the samples are not correlated. This helps to reduce correlation between samples. We will talk more about the update rule of *workers* later in this section.

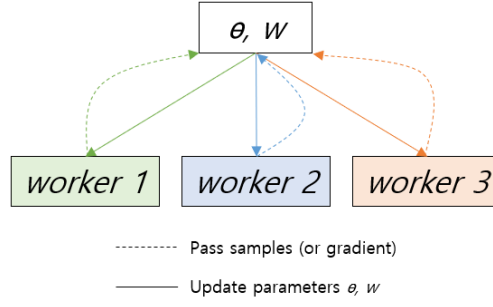


Figure 15: Workers in A3C algorithm

13.4.2 n -step TD

A2C algorithm used 1 -step TD when updating both actor and critic. A3C algorithm uses n -step TD (See section 9 for n -step TD). To derive n -step in A3C algorithm, we will begin from A2C algorithm. Let's see how 2 -step in A3C algorithm can be formulated. By doing so, we will be able to better understand n -step in A3C algorithm by expanding the idea of 2 -step A3C algorithm. The definition of Q could be expressed by the following.

$$Q(s_t, a_t) \triangleq \int_{s_{t+1}:a_\infty} G_t \cdot p(s_{t+1}, a_{t+1}, \dots | s_t, a_t) ds_{t+1} : a_\infty \quad (90)$$

By rewriting $p(s_{t+1}, a_{t+1}, \dots | s_t, a_t)$

$$p(s_{t+1}, a_{t+1}, \dots | s_t, a_t) = p(a_{t+2}, \dots | s_t, a_t, s_{t+1}, a_{t+1}, s_{t+2}) \cdot p(s_{t+1}, a_{t+1}, s_{t+2} | s_t, a_t) \quad (91)$$

$$= p(a_{t+2}, \dots | s_t, a_t, s_{t+1}, a_{t+1}, s_{t+2}) \cdot p(s_{t+2} | s_t, a_t, s_{t+1}, a_{t+1}) \cdot p(s_{t+1}, a_{t+1} | s_t, a_t) \quad (92)$$

$$= p(a_{t+2}, \dots | s_t, a_t, s_{t+1}, a_{t+1}, s_{t+2}) \cdot p(s_{t+2} | s_t, a_t, s_{t+1}, a_{t+1}) \cdot p(a_{t+1} | s_t, a_t, s_{t+1}) \cdot p(s_{t+1} | s_t, a_t) \quad (93)$$

$$= p(a_{t+2}, \dots | s_{t+2}) \cdot p(s_{t+2} | s_{t+1}, a_{t+1}) \cdot p(a_{t+1} | s_{t+1}) \cdot p(s_{t+1} | s_t, a_t) \quad (94)$$

and plugging in the last equation 94 back to the definition of Q , we get the following:

$$Q(s_t, a_t) \triangleq \int_{s_{t+1}:a_\infty} (R_t + \gamma R_{t+1} + \gamma^2 G_{t+2}) \cdot p(a_{t+2}, \dots | s_{t+2}) \cdot p(s_{t+2} | s_{t+1}, a_{t+1}) \cdot p(a_{t+1} | s_{t+1}) \cdot p(s_{t+1} | s_t, a_t) ds_{t+1} : a_\infty \quad (95)$$

$$= E_{s_{t+1}, a_{t+1}, s_{t+2}} [R_t + \gamma R_{t+1} + \gamma^2 V(s_{t+2}) | s_t, a_t] \quad (96)$$

In 2 -step A3C algorithm, the gradient of the objective function is (the following equation is imported from A2C algorithm)

$$\nabla_\theta J_\theta \approx \sum_{t=0}^{\infty} \int_{s_t, a_t, s_{t+1}} \nabla_\theta \ln p_\theta(a_t | s_t) \cdot (Q - V) \cdot p(s_{t+1} | s_t, a_t) \cdot p_\theta(a_t | s_t) \cdot p(s_t | s_{t-1}, a_{t-1}) ds_t, a_t, s_{t+1} \quad (97)$$

where

$$2\text{-step TD: } Q - V = E_{s_{t+1}, a_{t+1}, s_{t+2}} [R_t + \gamma R_{t+1} + \gamma^2 V(s_{t+2}) - V(s_t) | s_t, a_t] \quad (98)$$

The same rules applies to n -step TD. For example,

$$3\text{-step TD: } Q - V = E_{s_{t+1}, a_{t+1}, s_{t+2}, a_{t+2}, s_{t+3}} [R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + V(s_{t+3}) - V(s_t) | s_t, a_t] \quad (99)$$

$$4\text{-step TD: } Q - V = E_{s_{t+1}, a_{t+1}, s_{t+2}, a_{t+2}, s_{t+3}, a_{t+3}, s_{t+4}} [R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \gamma^3 R_{t+3} + \gamma^4 V(s_{t+4}) - V(s_t) | s_t, a_t] \quad (100)$$

$$\vdots \quad (101)$$

Let's take a simple example when $n = 3$ in A3C algorithm. The algorithm uses 1 -step, 2 -step, and 3 -step TD to update both actor and critic (See Figure 16).

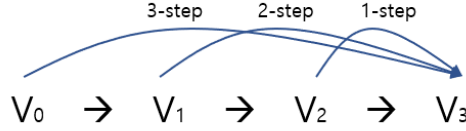


Figure 16: 3-step TD in A3C algorithm. When we are at $t = 2$, we can acquire V_3

$$3\text{-step TD: } R_0 + \gamma R_1 + \gamma^2 R_2 + \gamma^3 V_3 - V_0 = A_0 \quad (102)$$

$$2\text{-step TD: } R_1 + \gamma R_2 + \gamma^2 V_3 - V_1 = A_1 \quad (103)$$

$$1\text{-step TD: } R_2 + \gamma V_3 - V_2 = A_2 \quad (104)$$

We can define our new actor update function by using advantage function A . n is decided by N in A3C algorithm. When $N = 3$, the objective function and actor update can be formulated by the following, and A_i follows the previous equations 102, 103, 104.

$$\nabla_{\theta} J_{\theta} \approx \sum_{t=0}^{\infty} \int_{s_t, a_t, s_{t+1}} \nabla_{\theta} \ln p_{\theta}(a_t | s_t) \cdot (Q - V) \cdot p(s_{t+1} | s_t, a_t) \cdot p_{\theta}(a_t | s_t) \cdot p(s_t | s_{t-1}, a_{t-1}) ds_t, a_t, s_{t+1} \quad (105)$$

$$\text{Actor update: } \theta \leftarrow \theta + \alpha \cdot \nabla_{\theta} \sum_{i=t-2}^t \ln p_{\theta}(a_i | s_i) \cdot A_i \quad (106)$$

It is known that when N is large, it is high variance and low bias. In contrast, when N is small, it is low variance and high bias.

13.4.3 Entropy

The algorithm introduces entropy for actor and this does not apply to critic. This enables the actor to take various actions and allows exploration. The idea is straightforward. We add entropy of policy in actor update process.

$$\text{Actor update: } \theta \leftarrow \theta + \alpha \cdot \nabla_{\theta} \sum_{i=t-N+1}^t (\ln p_{\theta}(a_i | s_i) \cdot A_i + \lambda \cdot H_i(p_{\theta}(a_i | s_i))) \quad (107)$$

λ adjusts the effect of entropy. Assuming the policy is Gaussian distribution, the standard deviation of the policy is proportional to entropy. That is, the larger the standard deviation, the more diverse the actor can perform (See 14.3 for more details). We can visualize this by the following diagram. Actor update function now considers two terms:

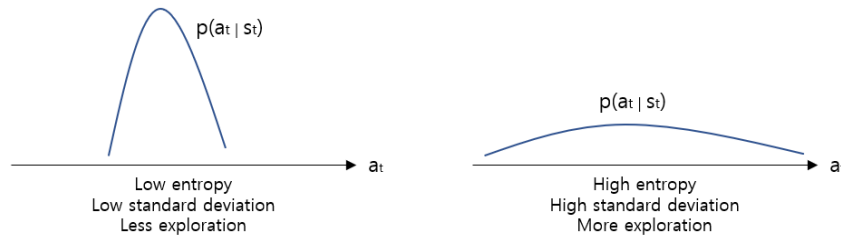


Figure 17: Relationship between entropy, standard deviation, and exploration

$\ln p_{\theta}(a_t | s_t) \cdot A_i$ and $H_i(p_{\theta}(a_i | s_i))$. When λ is large, the actor focuses on exploration. When it is small, the actor focuses on the first term.

13.4.4 Final algorithm

Asynchronous Advantage Actor-Critic algorithm

Algorithm parameters: actor(policy) parameter θ and critic parameter w ;
Algorithm hyper-parameters: maximum step T , discount factor γ , sample size N , learning rate for actor α , and critic β ;

Initialize θ and w . Choose maximum step T , discount factor γ , sample size N , learning rate α , and β ;
For an asynchronous worker,

repeat

 Collect N samples (single sample: s_t, a_t, R_t, s_{t+1});

 Actor update: $\theta \leftarrow \theta + \alpha \cdot \nabla_{\theta} \sum_{i=t-N+1}^t (\ln p_{\theta}(a_i|s_i) \cdot A_i + \lambda \cdot H_i(p_{\theta}(a_i|s_i)))$;

 Critic update: $w \leftarrow w - \beta \cdot \nabla_w \sum_{i=t-N+1}^t A_i^2$;

 Clear the batch (Remove N samples)

until we find satisfactory parameters: θ and w ;

Algorithm 9: Asynchronous Advantage Actor-Critic algorithm

13.4.5 Update rule of workers

There are two ways to update parameters θ and w . The first option is called **gradient parallelism**. In case of gradient parallelism, each worker calculates gradient of actor and critic using N samples. Afterwards, the worker passes the gradients to the center (global network). The global network then updates its own θ and w . The updated θ and w get copied into each worker back. This method is advantageous when expanding the number of workers because the gradients are calculated by the workers.

The second option is called **data parallelism**. In case of data parallelism, the global network calculates gradients when N samples are given from each worker. A single worker generates N samples and passes the generated samples to the global network. The worker is to just generate samples. Everything else is the global network's job. The global network updates its gradients and copies its gradients into each worker back. The global network calculates the gradients and updates the parameters.

- 13.5 Proximal Policy Optimization (PPO)**
- 13.6 Deep Deterministic Policy Gradient (DDPG)**
- 13.7 TD3**
- 13.8 SAC**

14 Additional Details

14.1 Appendix

- Logarithmic Properties:

- Product Rule

$$\log_a(xy) = \log_a x + \log_a y \quad (108)$$

- Quotient Rule

$$\log_a \frac{x}{y} = \log_a x - \log_a y \quad (109)$$

- Marginalization:

$$\int_x p(x, y) dx = p(y) \int_y p(x, y) dy = p(y) \quad (110)$$

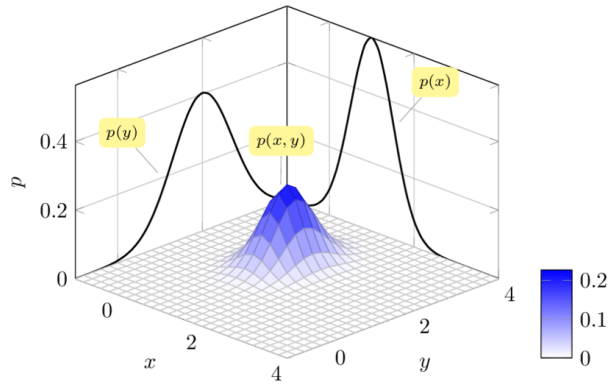


Figure 18: Visualising marginalization [2]

- Bayesian Rule:

$$p(x, y) = p(x|y) \cdot p(y) \quad (111)$$

$$p(x, y|z) = p(x|y, z) \cdot p(y|z) \quad (112)$$

- Law of large numbers (LLN):

$$E[x] \triangleq \int_x xp(x) dx \approx \frac{1}{N} \sum_{i=1}^N x_i, \quad x \sim p(x) \quad (113)$$

14.2 Simplification of the policy gradient equation

We will show how the equation 58 could be simplified into 59

$$(R_0 + \gamma R_1 + \gamma^2 R_2 + \dots) \cdot (\nabla_\theta \sum_{t=0}^{\infty} \ln p_\theta(a_t|s_t)) \quad (114)$$

We can expand the above equation to the following:

$$(R_0 + \gamma R_1 + \gamma^2 R_2 + \dots) \cdot (\nabla_\theta \ln p_\theta(a_0|s_0) + \nabla_\theta \ln p_\theta(a_1|s_1) + \nabla_\theta \ln p_\theta(a_2|s_2) + \dots) \quad (115)$$

It turns out that when we multiply the reward and policy, and if the index of the reward is smaller than the index of the policy, then the multiplied term becomes 0. That is, the following holds when p is smaller than q .

$$\int_\tau \gamma^p R_p \cdot \nabla_\theta \ln p_\theta(a_q|s_q) \cdot p_\theta(\tau) d\tau = 0 \quad (116)$$

For instance,

$$\int_{\tau} R_0 \cdot \nabla_{\theta} \ln p_{\theta}(a_1|s_1) \cdot p_{\theta}(\tau) d\tau \quad (117)$$

$$= \int_{\tau} R_0 \cdot \nabla_{\theta} \ln p_{\theta}(a_1|s_1) \cdot p_{\theta}(a_1|\tau_{-a_1}) p(\tau_{-a_1}) d\tau \quad (118)$$

$$= \int_{\tau} R_0 \cdot \nabla_{\theta} \ln p_{\theta}(a_1|s_1) \cdot p_{\theta}(a_1|s_1) p(\tau_{-a_1}) d\tau \quad (119)$$

$$= \int_{\tau_{-a_1}} R_0 \int_{a_1} \nabla_{\theta} \ln p_{\theta}(a_1|s_1) p_{\theta}(a_1|s_1) da_1 p(\tau_{-a_1}) d\tau_{-a_1} \quad (120)$$

$$= \int_{\tau_{-a_1}} R_0 \int_{a_1} \frac{\nabla_{\theta} p_{\theta}(a_1|s_1)}{p_{\theta}(a_1|s_1)} p_{\theta}(a_1|s_1) da_1 p(\tau_{-a_1}) d\tau_{-a_1} \quad (121)$$

$$= \int_{\tau_{-a_1}} R_0 \int_{a_1} \nabla_{\theta} p_{\theta}(a_1|s_1) da_1 p(\tau_{-a_1}) d\tau_{-a_1} \quad (122)$$

$$= \int_{\tau_{-a_1}} R_0 \left(\nabla_{\theta} \int_{a_1} p_{\theta}(a_1|s_1) da_1 \right) p(\tau_{-a_1}) d\tau_{-a_1} \quad (123)$$

$$= \int_{\tau_{-a_1}} R_0 (\nabla_{\theta} 1) p(\tau_{-a_1}) d\tau_{-a_1} \quad (124)$$

$$= \int_{\tau_{-a_1}} R_0 \cdot 0 \cdot p(\tau_{-a_1}) d\tau_{-a_1} \quad (125)$$

$$= 0 \quad (126)$$

Therefore, we can rewrite the equation 114 to the following:

$$\sum_{t=0}^{\infty} (\nabla_{\theta} \ln p_{\theta}(a_t|s_t)) \cdot \left(\sum_{k=t}^{\infty} \gamma^k \cdot R_k \right) \quad (127)$$

14.3 Entropy and Gaussian distribution:

We will show why standard deviation of the policy is proportional to entropy. Entropy H of $p(x)$ can be defined by the following assuming that $p(x)$ follows Gaussian distribution.

$$H(p(x)) \triangleq - \int p(x) \ln p(x) dx \quad (128)$$

$$\text{where } p(x) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right) \quad (129)$$

$$H(p(x)) = - \int p(x) \ln p(x) dx \quad (130)$$

$$= - \int p(x) \ln \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right) dx \quad (131)$$

$$= \int p(x) \cdot \ln \sqrt{2\pi}\sigma^2 dx + \int p(x) \cdot \frac{(x-\mu)^2}{2\sigma^2} dx \quad (132)$$

$$= \frac{1}{2} (1 + \ln 2\pi\sigma^2) \quad (133)$$

15 Check your understanding

1. How many policies there are when there are 7 discrete states and 2 actions per state?
→ 2^7 . Generally, the number of policies is $|A|^{|S|}$.
2. Is the optimal policy for a MDP always unique?
→ No.
- 3.

References

- [1] Understanding the role of the discount factor in reinforcement learning. <https://stats.stackexchange.com/questions/221402/understanding-the-role-of-the-discount-factor-in-reinforcement-learning>, 2016.
- [2] Raghavendra Selvan. *Bayesian tracking of multiple point targets using Expectation Maximization*. PhD thesis, 07 2015.